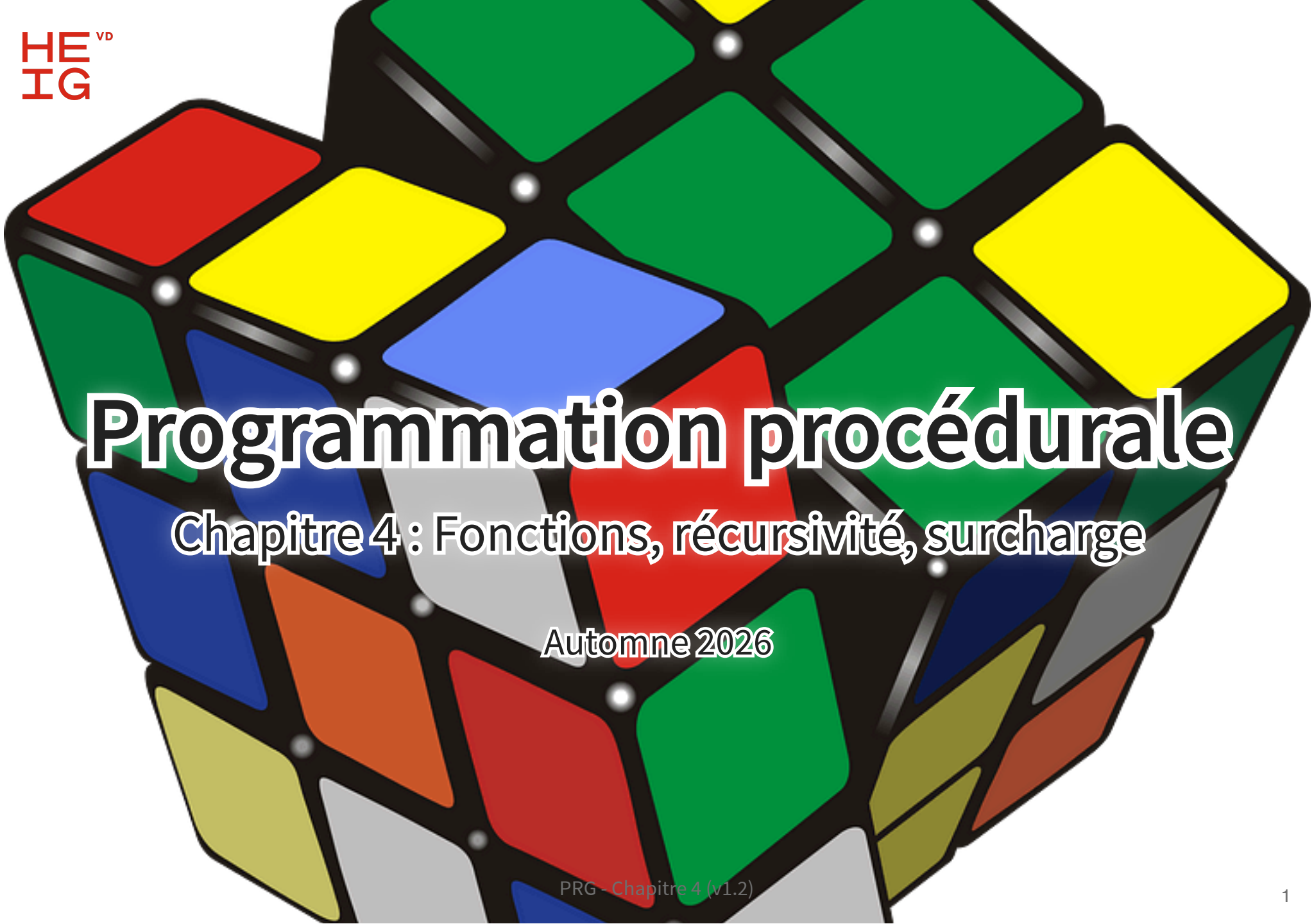


Programmation procédurale

Chapitre 4 : Fonctions, récursivité, surcharge

Automne 2026

Structure du chapitre



1. Introduction
2. Passage des paramètres : par valeur, par référence, par adresse
3. Types des paramètres lors des appels
4. `return`
5. Récursivité
6. Prototypes et compilation séparée
7. Variables locales et globales
8. Éléments pour une bonne conception du code
9. Surcharge de fonctions



1. Introduction

Pourquoi utiliser des fonctions ?



- Quand un programme dépasse quelques pages de texte, il est pratique de le **décomposer** en parties :
  - relativement **indépendantes**
  - dont on peut **comprendre** le rôle sans avoir à examiner l'ensemble du code
- La **programmation procédurale** permet un premier pas dans ce sens grâce à la notion de fonction, i.e. :
  - un **bloc d'instructions** auquel on donne un **nom**
  - que l'on peut **appeler** depuis un autre point du code
  - éventuellement **plusieurs fois**
  - éventuellement en lui fournissant des **paramètres**

Fonctions mathématiques vs fonctions C++



- En **mathématiques**, une fonction :
  - possède des **arguments** dont on fournit la valeur à l'appel : `sqrt(x)`
  - fournit un **résultat scalaire** désigné simplement par son appel : `sqrt(x)` désigne le résultat, que l'on peut utiliser dans `y + 2 * sqrt(x)`.
- En **C++**, une fonction peut :
  - **modifier** les valeurs de certains **paramètres** transmis
  - réaliser une **action** autre qu'un simple calcul
  - fournir un **résultat non scalaire** (string, structures, objets)
  - fournir une valeur résultat que **l'on n'utilise pas**
  - **ne pas fournir de résultat** du tout ; dans certains langages (Pascal, Ada), on parle alors de **procédure**.

Exemple : la fonction pow



- Nous avons déjà utilisé des fonctions. Par exemple, `<cmath>` nous fournit la fonction `pow` qui prend deux paramètres : la base et l'exposant.
- Le code ci-contre affiche x^n et y^n pour $x = 2$, $y = \frac{1}{2}$ et $n = 10$, en appelant deux fois la fonction `pow`.

```cpp
#include <cmath>
#include <cstdlib>
#include <iostream>
using namespace std;

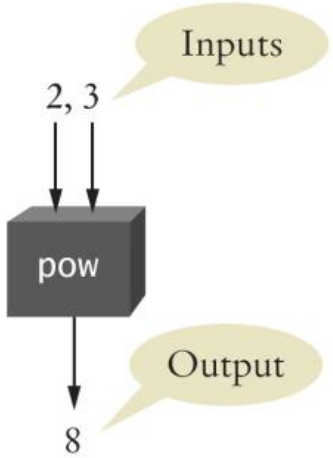
int main() {
   double n = 10.0;
   double x = 2.0, y = 0.5;

   cout << pow(x, n) << endl;
   cout << pow(y, n) << endl;
}
```

```
1024
0.000976562
```

Une boîte noire

- Nous n'avons **pas besoin de savoir comment** le calcul de la puissance est effectué.
- Nous pouvons considérer cette fonction comme une **boîte noire** dont le concepteur nous **garantit** que :
  - si nous lui fournissons **en entrée** les paramètres `base` et `exposant`
  - elle nous fournit **en retour** la valeur $base^{exposant}$



Notre propre fonction puissance



- **Réalisons nous-mêmes** une fonction `puissance` simplifiée en ne considérant que des exposants entiers ≥ 0.
- Le code permettant de calculer $val = 2^{10}$ est assez simple :

```
double base     = 2.0;
int    exposant = 10;

double val = 1;

for (int i = 0; i < exposant; ++i) {
   val *= base;
}
```

- valeurs d'entrée
- résultat à calculer
- instructions effectuant le calcul

- Il faut maintenant le mettre en forme pour qu'il soit **appelable** via `puissance(2.0, 10)`.

Notre propre fonction puissance



- Mis sous la forme d'une fonction, ce code devient :

```
double puissance(double base, int exposant)
{
    double val = 1;

    for (int i = 0; i < exposant; ++i) {
        val *= base;
    }

    return val;
}
```

- nom
- paramètres
- type de la valeur de retour
- corps de la fonction
- déclaration d'une variable locale
- instructions
- retour de la valeur résultat

La fonction main()



- Notons que depuis notre tout premier programme, nous écrivons une fonction, la fonction `main` :

```
int main() {
   …
   return EXIT_SUCCESS;
}
```

- Elle ne prend pas de **paramètres** et retourne une valeur entière. Elle garantit de **retourner EXIT_SUCCESS** (qui correspond à 0) si tout se passe bien, et une valeur non nulle sinon.
- Il en existe aussi une version avec des paramètres d'entrée (voir le cours PAY) :

```
int main(int argc, const char* argv[]);
```

void

- Une fonction peut aussi ne pas fournir de valeur en retour.
- On l'indique en utilisant le type `void` comme type de retour :

```
void afficher(int val, int width) {
   cout << "|" << setw(width) << val << "|" << endl;
}

int main() {
   afficher(1, 6);
   afficher(2, 6);
   return EXIT_SUCCESS;
}
```

```
|     1|
|     2|
```



2. Passage des paramètres

- a) par valeur
- b) par référence
- c) par adresse

Passage des paramètres



- Les paramètres permettent de **transmettre des informations entre le code** appelant la fonction **et la fonction** elle-même.
- On peut **transmettre** ces paramètres :
  - **par valeur** : copie une variable du code appelant dans une variable de la fonction
  - **par référence** : accès à une variable du code appelant depuis la fonction
  - **par adresse** : accès à une variable du code appelant depuis la fonction, par son adresse.
- NB : dans les deux derniers cas, un passage **constant** est possible.

Passage par valeur

```
void echanger(int a, int b) {
   int t;
   cout << "debut echange: a = " << a << " , b = " << b << endl;
   t = a; a = b; b = t;
   cout << "fin echange  : a = " << a << " , b = " << b << endl;
}

int main() {
   int c = 3, d = 5;
   cout << "avant echange: c = " << c << " , d = " << d << endl;
   echanger(c, d);
   cout << "apres echange: c = " << c << " , d = " << d << endl;
}
```

- Qu'affiche ce code ?

```
avant echange: c = 3 , d = 5
debut echange: a = 3 , b = 5
fin echange  : a = 5 , b = 3
apres echange: c = 3 , d = 5
```

Passage par valeur



- Dans ce code, les paramètres `a` et `b` sont **passés par valeur** :

```
void echanger(int a, int b) {…}
```

- `a` et `b` sont des **variables locales** de la fonction `echanger`, initialisées à l'appel de la fonction en **copiant** les valeurs transmises par le code appelant.
- Ces valeurs et leurs types sont éventuellement **convertis** à l'appel, en suivant les mêmes règles que pour l'opérateur d'affectation.
- **Modifier** les valeurs de `a` ou `b` **dans la fonction** n'a **aucun effet** sur les valeurs dans le code appelant.
- Le passage par valeur permet uniquement de transmettre des paramètres **en entrée, pas en sortie !**

Passage par valeur constante



- Comme pour les variables, on peut utiliser le mot `const` pour indiquer qu'un paramètre n'est **pas modifié par le code de la fonction**.
- Note : pour les paramètres passés **par valeur, cela ne change rien** du point de vue de l'appelant, donc il n'est pas utile d'utiliser `const` dans ce cas.

```
void f(const int a, const int b) {

   a = a * 2;   // ne compile pas !

   cout << a << ' ' << b << endl;
}
```

Passage par référence

```
void echanger(int& a, int& b) {
   int t;
   cout << "debut echange: a = " << a << " , b = " << b << endl;
   t = a; a = b; b = t;
   cout << "fin echange  : a = " << a << " , b = " << b << endl;
}

int main() {
   int c = 3, d = 5;
   cout << "avant echange: c = " << c << " , d = " << d << endl;
   echanger(c, d);
   cout << "apres echange: c = " << c << " , d = " << d << endl;
   return EXIT_SUCCESS;
}
```

- Qu'affiche ce code ?

```
avant echange: c = 3 , d = 5
debut echange: a = 3 , b = 5
fin echange  : a = 5 , b = 3
apres echange: c = 5 , d = 3
```

Passage par référence



- Pour pouvoir modifier des variables du code appelant depuis une fonction, il faut utiliser un autre mécanisme : le **passage par référence**.
- On l'indique en ajoutant le caractère **&** au **type** du paramètre.
- Le paramètre est un **synonyme** de la variable passée par le code appelant, donc modifier ce paramètre **modifie la variable dans le code appelant**.
- Le **passage par référence** permet donc d'utiliser les paramètres **en entrée ou en sortie**.
- **Attention** : le paramètre effectif (lors de l'appel) doit être **une variable** et doit être **du même type** que celui du paramètre formel.

Passage par référence constante



- Comme pour les variables, on peut utiliser le mot `const` pour indiquer qu'un paramètre n'est **pas modifié par le code de la fonction**.
- Pour les paramètres **passés par référence**, cela a deux impacts majeurs pour l'appelant :
  - Il sait que la **variable** passée par référence ne sera **pas modifiée**
  - Il ne subit **pas les restrictions d'appel (variable et type)** du passage par référence : il peut y avoir une **conversion implicite si nécessaire**.

Adresse d'une valeur



- Avant de reproduire l'exemple précédent en utilisant le passage par adresse, prenons le temps de comprendre de quoi il s'agit.
- Les **constantes** et **variables** sont **stockées dans la mémoire physique** de l'ordinateur. Elles ont donc nécessairement une **adresse**.
- Cette adresse ne sera **pas la même d'une exécution à l'autre**, elle n'est d'ailleurs pas connue à la compilation.
- L'utilisation d'un identificateur est plus pratique et plus lisible.

Adresse d'une valeur

- Une variable (ou constante) peut simplement être utilisée par son identificateur :

```
int valeur = 3;
cout << " valeur : " <<  valeur;  //  valeur : 3
```

- Pour connaître l'adresse d'une variable (ou constante), il faut utiliser l'opérateur **&** qui retourne son adresse :

```
cout << "&valeur : " << &valeur;  // &valeur : 0x7ff7ba4e46f8
```

Adresse d'une valeur



- Pour recevoir une adresse en paramètre, il faut la stocker, ce qui implique un type particulier. Ce sera une adresse sur … le type de la valeur, soit **type*** :

```
int* adr = &valeur;
```

- Sur la base d'une adresse, comment lire ou écrire une valeur ? Il faut déréférencer cette adresse avec l'opérateur * :

```
cout << "*adr : " << *adr << endl; // *adr : 3
```

- **IMPORTANT** : `type*` et `*adr`, ce n'est pas le même opérateur 🧐 :
  - **type*** est lié à une **déclaration**
  - ***adr** est un opérateur utilisé dans une **instruction**

Adresse d'une valeur

- Pour finir, l'opérateur & retourne l'adresse d'une variable ou d'une constante. Voyons tout ceci dans le cadre d'une fonction :

```
void adresse(int* adr) {
   cout << " adr    : " <<  adr << endl;
   cout << "*adr    : " << *adr << endl;
   cout << "&adr    : " << &adr << endl;
}
int main() {
   int valeur = 3;
   cout << " valeur : " <<  valeur << endl;
   cout << "&valeur : " << &valeur << endl;
   adresse(&valeur);
}
```

```
 valeur : 3
&valeur : 0x16cf7f568
 adr    : 0x16cf7f568
*adr    : 3
&adr    : 0x16cf7f4d8
```

- Note : dans notre fonction, le paramètre `adr` contient une adresse … et est lui-même stocké à une adresse ;)

Passage par adresse

```
void echanger(int* a, int* b) {
   int t;
   cout << "debut echange: a = " << *a << " , b = " << *b << endl;
   t = *a; *a = *b; *b = t;
   cout << "fin echange  : a = " << *a << " , b = " << *b << endl;
}

int main() {
   int c = 3, d = 5;
   cout << "avant echange: c = " << c << " , d = " << d << endl;
   echanger(&c, &d);
   cout << "apres echange: c = " << c << " , d = " << d << endl;
}
```

- Qu'affiche ce code ?

```
avant echange: c = 3 , d = 5
debut echange: a = 3 , b = 5
fin echange  : a = 5 , b = 3
apres echange: c = 5 , d = 3
```

Passage par adresse

Pour mémoire …

- Pour des raisons de sécurité, **une adresse est typée** (`int*`, `string*`, …). Le pointeur doit être du type de la valeur pointée, sous peine d'erreur de compilation :

```
int    entier  = 3;
double reel    = 2.7;
int*   ptr = &reel;
```

```
error: cannot initialize a variable of type 'int *'
       with an rvalue of type 'double *'
```

- Un pointeur peut traiter la valeur pointée comme constante (accès en lecture seule) :

```
const int CSTE    = 2;
      int valeur  = 2;
const int* adr1 = &valeur; // accès en lecture seule sur une variable
      int* adr2 = &valeur; // accès en lecture/écriture sur une variable
const int* adr3 = &CSTE;   // accès en lecture seule sur une constante
      int* adr4 = &CSTE;   // accès en écriture sur une constante ?
```

```
error: cannot initialize a variable of type 'int *' with an rvalue of type 'const int *'
```



3. Types des paramètres lors des appels

Conversion des types lors du passage par valeur



```
void afficher(int p1, char p2) {
   cout << "p1 = " << p1 << " , p2 = " << p2 << endl;
}

int main() {
   const char CAR = 'A';
   afficher(1, 'A');
   afficher(1, 65);
   afficher('A', 67);
   afficher(CAR, CAR + 1);
   return EXIT_SUCCESS;
}
```

- Qu'affiche ce code ?

```
p1 = 1 , p2 = A
p1 = 1 , p2 = A
p1 = 65 , p2 = C
p1 = 65 , p2 = B
```

Usage du passage par référence constante



- **Pourquoi** passer par référence constante plutôt que par valeur ?
  - Pour les types **simples**, cela n'a pas vraiment d'intérêt
  - Pour les types **composés** (string, tableaux, classes créées par vous), on **économise une copie** (si pas de conversion) qui peut être coûteuse.
- **Pourquoi** passer par référence constante plutôt que par référence simple ?
  - Une référence simple exige une variable modifiable du bon type (ni littéral, ni expression, ni conversion).
  - Une référence constante accepte tout cela et garantit de ne pas modifier la variable :

```
void afficher(string message) { … } ✘
void afficher(string& message) { … } ✘
void afficher(const string& message) { … } ✔
```



4. return

Valeur de retour

- Le mot-clé **`return`** fournit le résultat à l'appelant via la syntaxe : **`return`** `EXPRESSION;`
- Il **interrompt l'exécution de la fonction**, de manière similaire à **`break`** pour une boucle.
- Si nécessaire, `EXPRESSION` est convertie dans le type annoncé comme résultat de la fonction :

```
bool estImpair(int valeur) {
   // implicitement
   return valeur % 2;
}
```

```
bool estImpair(int valeur) {
   // explicitement
   return bool(valeur % 2);
}
```

Valeur de retour

- Les instructions placées **après un `return` ne sont jamais exécutées**.
- Plusieurs **`return`** peuvent être présents, en général **soumis à des conditions**.
- **À utiliser intelligemment**.

```
bool estImpair(int valeur) {
   return valeur % 2;
   …
   cout << "inutile";
   …
}
```

```
int valeur(char car) {
   switch (car) {
      case '0': return  0;
      …
      case '9': return  9;
      default : return -1;
   } // switch
}
```

Valeur de retour



- On peut aussi utiliser **`return`** sans `EXPRESSION` selon la syntaxe : **`return;`**
- L'exécution est **interrompue** et **aucune valeur** n'est renvoyée.
- Ce comportement est **correct** si le type de retour est `void`, **sinon** le comportement est **indéfini**.
- Si l'on **atteint l'accolade `}`** de fin de corps de fonction sans avoir rencontré de **`return`**, c'est ce **`return`** sans `EXPRESSION` qui est **utilisé implicitement**.
- Une **exception** à cette règle : pour **la fonction principale** `main`, la ligne **`return 0;`** est ajoutée implicitement en fin de fonction.

Retour par référence

- Notons que l'on peut **retourner une référence**.
- Cela permet d'utiliser ce résultat **à gauche** d'une affectation.
- Utile quand c'est la fonction qui choisit la variable à modifier : `plusPetit(score1, score2) += 5` donne un bonus au plus faible des deux scores.
- C'est aussi ce qui permet de modifier un élément d'un conteneur : `v[1] = 42` fonctionne parce que l'opérateur `[]` de `vector` retourne une référence (chapitre 5).
- Ne **pas retourner de référence** à une **variable locale**, mais uniquement :
  - à une variable globale
  - à une référence passée en paramètre
  - à une variable allouée dynamiquement

```
int& plusPetit(int& a, int& b) {
   return a < b ? a : b;
}

int main() {
   int  a = 5;
   int  b = 7;
   int  c = plusPetit(a, b);
   int& d = plusPetit(a, b);
   plusPetit(a, b) = 12;
   d = 10;

   cout << "a = " << a
      << ", b = " << b
      << ", c = " << c;
}
```

```
a = 10, b = 7, c = 5
```



5. Récursivité

Récursivité



- Parfois des problèmes sont plus simples à exprimer de manière **récursive**.
- Exemple : factorielle de n, notée n! :
  - 5! = 1 × 2 × 3 × 4 × 5
  - 6! = 1 × 2 × 3 × 4 × 5 × 6, donc 6! = 5! × 6
  - **n! = (n - 1)! × n** avec n ≥ 0

```
int facto(int n) {

   // cas trivial
   if (n <= 1)
      return 1;

   // appel récursif
   return n * facto( n - 1 );
}
```

Récursivité



- Il est essentiel de respecter des conditions d'implémentation :
  - Une fonction récursive s'**appelle elle-même** : `return n * facto(...);`
  - Une **condition d'arrêt** stoppe les appels récursifs et, ici, retourne une valeur : `if (n <= 1) return 1;`
  - Les paramètres tendent **vers la condition d'arrêt** : `return n * facto( n - 1 );`
- Une fonction récursive est en général **très courte**.

Récursivité



- Il est souvent préférable d'opter pour une **implémentation itérative** lorsque celle-ci est possible sans être trop compliquée ;)

```
int facto(int n) {

   int r = 1;

   for (; n > 1; --n) {
      r *= n;
   }

   return r;
}
```

Récursivité



- Un autre exemple très connu est la **suite de Fibonacci** :

n	0	1	2	3	4	5	6	7	8	9	10	11	12	…
f(n)	0	1	1	2	3	5	8	13	21	34	55	89	144	…

- Ou plus simplement :

$$f(0) = 0 \quad f(1) = 1$$

$$f(n) = f(n-2) + f(n-1)$$

```
int fibo(int n) {
   // cas trivial
   if (n < 2)
      return n;

   // appels récursifs
   return fibo(n-2) + fibo(n-1);
}
```

- Ce sujet sera repris en détail dans le cours d'ASD : Algorithmes et Structures de Données.



6. Prototypes et compilation séparée

Prototypes



- Tout comme on ne peut utiliser une variable avant de l'avoir déclarée, on ne **peut pas appeler une fonction avant de l'avoir déclarée**.

Le code suivant ne compile pas :

```c
int main() {
   int a = f(0);
}

int f(int val) {
   return val + 42;
}
```

```
error: use of undeclared identifier
'f'
```

Une solution simple consiste à définir la fonction avant de l'utiliser :

```c
int f(int val) {
   return val + 42;
}

int main() {
   int a = f(0);
}
```

La solution simple ne suffit pas



- La solution « simple » est dans les faits compliquée à mettre en œuvre.
- Le programmeur doit trier les fonctions dans un ordre qui garantit que toute fonction utilisée par une autre soit placée avant celle-ci.
- Ce tri est impossible dans le cas ci-contre où `f()` appelle `g()` et vice-versa : le code ne compile pas.
- Avec le prototype de la diapo suivante, il compile et affiche 511.

```
int f(int x) {
   if(x <= 0) return x;
   return g(x-1) + 1;
}

int g(int x) {
   if(x <= 0) return x;
   return f(x / 2) * 2;
}

int main() {
   cout << f(1000) << endl;
}
```

```
error: use of undeclared
identifier 'g'
```

Prototypes



- La solution propre consiste à **séparer** :
  - la **déclaration** de la fonction, son **prototype**, qui exprime l'interface qu'elle propose à ses appelants.
  - de la **définition** de la fonction, qui indique comment la fonction est mise en œuvre :

```c
int f(int val);    // déclaration = prototype

int main() {
   int a = f(0);
}

int f(int val) {   // définition
   return val + 42;
}
```

HEIG VD



Prototypes

- Dans le prototype d'une fonction, les **noms des paramètres** sont **facultatifs** :
  - Ils peuvent être **omis** ou **différer** des noms utilisés dans la définition … **mauvaise pratique !**
  - Ils servent uniquement à **documenter** le code, ce qui est essentiel
- Les deux lignes suivantes sont donc équivalentes :

```
int f(int val1, double val2);
    int f(int, double);
```

- Ce qui importe pour le compilateur, c'est uniquement :
  - le **type** des paramètres
  - leur **ordre**

Valeurs par défaut des paramètres

- Le prototype, **mais pas la définition**, peut également préciser **des valeurs par défaut** pour certains paramètres.
- Cela permet d'appeler la fonction avec une **liste de paramètres réduite**, en omettant un ou plusieurs des **paramètres les plus à droite** :

```
void f(int i1, int i2 = 42, double d = 0.);

void f(int i1, int i2, double d) {
   cout << i1 << " " << i2 << " " << d << endl;
}

int main() {
   f(10, 20, 30);
   f(10, 20);
   f(10);
}
```

```
10 20 30
10 20 0
10 42 0
```

Compilation séparée



- La **séparation** entre **prototype** et **définition** des fonctions permet de **découper le code en plusieurs fichiers** :

```
int f(int val);

int main() {
   int a = f(0);
}

int f(int val) {
   return val + 42;
}
```

maFonction.h

```
int f(int val);
```

main.cpp

```
#include "maFonction.h"
int main() {
   int a = f(0);
}
```

maFonction.cpp

```
#include "maFonction.h"
int f(int val) {
   return val + 42;
}
```

Compilation séparée



- Les déclarations de **prototypes** sont situées dans des **fichiers d'en-tête** (header, d'où l'extension **.h** ou **.hpp**).
- On peut **inclure** ces fichiers avec la directive de compilation `#include`, suivie du nom de fichier entre `""`, ce qui les distingue des en-têtes de la bibliothèque standard que l'on entoure de **<>**.
- Les **définitions** de fonctions, y compris de la fonction principale `main`, sont situées dans des fichiers `.cpp`.
- Chaque fichier `.cpp` inclut les fichiers d'en-tête déclarant les fonctions qu'il :
  - **utilise**
  - **définit**

Compilation séparée



- La **compilation** s'effectue **en deux étapes** :
  - Le compilateur compile chaque fichier `.cpp` séparément pour créer un **module objet** (`.o` ou `.obj`) :
    - toutes les fonctions utilisées par un fichier `.cpp` doivent être **déclarées avant d'être utilisées**.
    - les fonctions utilisées par un fichier n'ont **pas besoin d'y être définies**
  - L'éditeur de liens regroupe ces modules objets ainsi que ceux de la bibliothèque standard dont il a besoin :
    - toutes les fonctions utilisées **doivent être définies**
    - une et **une seule fois**

#include imbriqués



- Les fichiers d'en-tête peuvent eux-mêmes inclure d'autres fichiers d'en-tête. Par exemple, si votre en-tête déclare la fonction `void afficher(const std::string& s);`, votre fichier doit contenir avant cette déclaration la ligne `#include <string>`, qui déclare le type `std::string` avant que vous ne l'utilisiez.
- Mais comment éviter les boucles d'inclusion infinies ?

header1.h

```
#include "header2.h"
…
```

header2.h

```
#include "header1.h"
…
```

#define, #ifdef, #ifndef, #else, #endif



- Solution : utiliser des **directives du préprocesseur** :
  - **`#ifndef`** inclut ce qui suit jusqu'à la directive *`#endif`* si le symbole n'est pas défini.
  - *`#define`* définit le symbole qui suit
- La première fois que le fichier est inclus, le symbole n'est pas défini et tout le texte entre **`#ifndef`** et *`#endif`* est inclus.
- S'il est inclus d'autres fois, le symbole est déjà défini et le texte entre **`#ifndef`** et *`#endif`* est ignoré.

maFonction.h

```
#ifndef MAFONCTION_H
#define MAFONCTION_H

int f(int val);

#endif
```

Attention, le symbole doit être unique. L'IDE vous suggère normalement un nom basé sur le nom du fichier d'en-tête et éventuellement de votre projet.



7. Variables locales et globales

HE VD IG



Variables locales

- Comme pour tout bloc d'instructions, on peut déclarer des variables dans le corps d'une fonction.
- Ces **variables locales** :
  - ne sont **visibles** que depuis l'intérieur de la fonction
  - **cachent** éventuellement des variables de même nom déclarées ailleurs
  - sont **créées automatiquement**, à chaque fois que l'on appelle la fonction
  - **disparaissent automatiquement**, à chaque fois que l'on sort de la fonction
- Ces propriétés nous aident à faire de nos fonctions des **boîtes noires**.

Variables globales



- Il est également possible de **déclarer** des **variables en dehors de tout bloc** et de toute fonction.
- Ces variables sont appelées **globales** et elles :
  - sont **visibles depuis tout le code** figurant **après** la déclaration de la variable globale.
  - peuvent être cachées par une variable locale du même nom
  - sont **créées statiquement**, une seule fois en début de programme, et sont **initialisées à zéro** (`0`, `0.`, `false`, `'\0'`) par défaut.
  - **ne disparaissent qu'à la fin de l'exécution** du programme

Variables globales



- L'utilisation de variables globales est en général une **mauvaise pratique**.
- En effet, une variable globale **peut être modifiée** par toutes les fonctions, ce qui rend sa valeur difficile à prédire et **est contraire au concept de boîte noire**.
- La librairie standard définit cependant quelques variables globales, telles que les « flux » nommés `cin`, `cout`, `cerr`, … car :
  - il n'y a qu'un seul de ces flux dans un programme
  - il doit être accessible depuis toutes les fonctions



8. Éléments pour une bonne conception du code

Éviter la duplication de code



- Quand un code est **répété presque à l'identique**, il faut sans doute le **remplacer par une fonction** :

```
int heures;
do {
   cout << "Entrez un entier entre 0 et 23: ";
   cin >> heures;
} while (heures < 0 || heures  > 23);



int minutes;
do {
   cout << "Entrez un entier entre 0 et 59: ";
   cin >> minutes;
} while (minutes < 0 || minutes > 59);
```

Éviter la duplication de code

- Si l'on occulte les différences, l'essentiel du code des 2 boucles est identique, et les parties occultées devront être des paramètres ou valeurs de retour de la fonction :

```
int heures;
do {
   cout << "Entrez un entier entre 0 et __: ";
   cin >> ________;
} while (________ < 0 || ________ > __);


int minutes;
do {
   cout << "Entrez un entier entre 0 et __: ";
   cin >> ________;
} while (________ < 0 || ________ > __);
```

Éviter la duplication de code



- Par ailleurs, on note que `23` et `59` sont des entrées des blocs, tandis que `heures` et `minutes` en sont des sorties :

```
int heures;
do {
   cout << "Entrez un entier entre 0 et 23: ";
   cin >> heures;
} while (heures < 0 || heures  > 23);



int minutes;
do {
   cout << "Entrez un entier entre 0 et 59: ";
   cin >> minutes;
} while (minutes < 0 || minutes > 59);
```

Éviter la duplication de code

- On obtient donc la fonction :

```
int lireUnEntierJusquA(int maxVal) {
   int input;
   do {
      cout << "Entrez un entier entre 0 et " << maxVal << ": ";
      cin >> input;
   } while (input < 0 || input > maxVal);
   return input;
}
```

- Avec les appels correspondants :

```
const int heures  = lireUnEntierJusquA(23);
const int minutes = lireUnEntierJusquA(59);
```

Éviter la duplication de code



- Mais on peut sans doute rendre cette fonction encore **plus réutilisable** :

```
int lireUnEntierEntre(int minVal, int maxVal) {
   int input;
   do {
      cout << "Entrez un entier entre " << minVal << " et " << maxVal << ": ";
      cin >> input;
   } while (input < minVal or input > maxVal);
   return input;
}
```

```
const int heures  = lireUnEntierEntre(0, 23);
const int minutes = lireUnEntierEntre(0, 59);
const int mois    = lireUnEntierEntre(1, 12);
```

Approche descendante : raffinement



Décomposer un problème complexe

en tâches plus simples

(et pour les tâches où c'est nécessaire, les décomposer en tâches plus simples)

(et pour les tâches où c'est nécessaire, les décomposer en tâches plus simples)

(et pour les tâches où c'est nécessaire, les décomposer en tâches plus simples)

(et pour les tâches où c'est nécessaire, les décomposer en tâches plus simples)

(et pour les tâches où c'est nécessaire, les décomposer en tâches plus simples)

Jusqu'à ce que les tâches soient réellement simples à mettre en œuvre

Approche descendante



Combien de jours séparent deux dates, par exemple du 12 mars au 5 octobre 2026 ?

La question est simple … mais en y réfléchissant :

- Tous les mois n'ont pas le même nombre de jours.
- Comment identifier une année bissextile ?
- Les saisies utilisateur sont multiples.
- Comment gérer les dates erronées à la saisie (30 février 2020) ?

Bien poser un algorithme et identifier les fonctions nécessaires est essentiel : on part du problème entier, puis on le découpe.

Calculer
nbre de jours
entre 2 dates

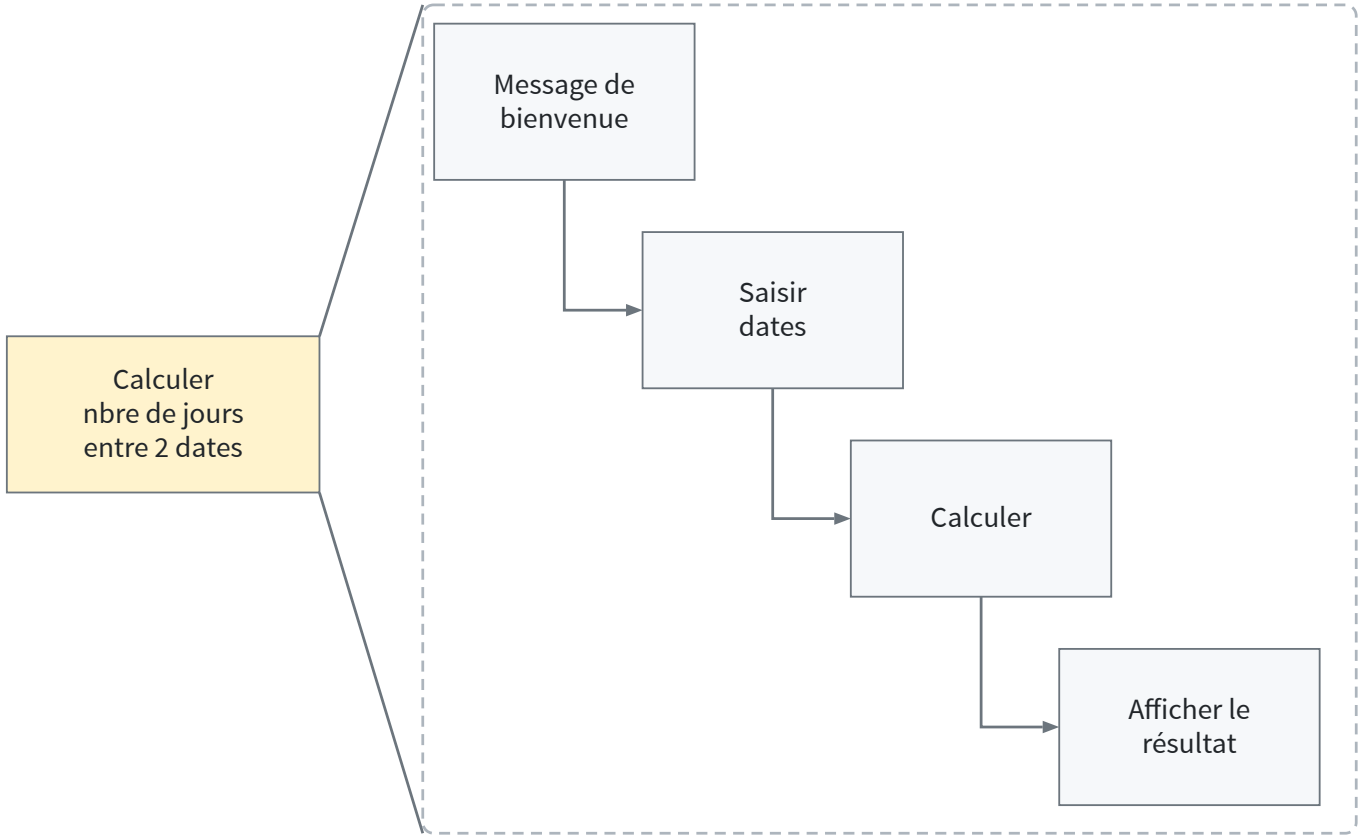
Mars 2026

L	M	M	J	V	S	D
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31					

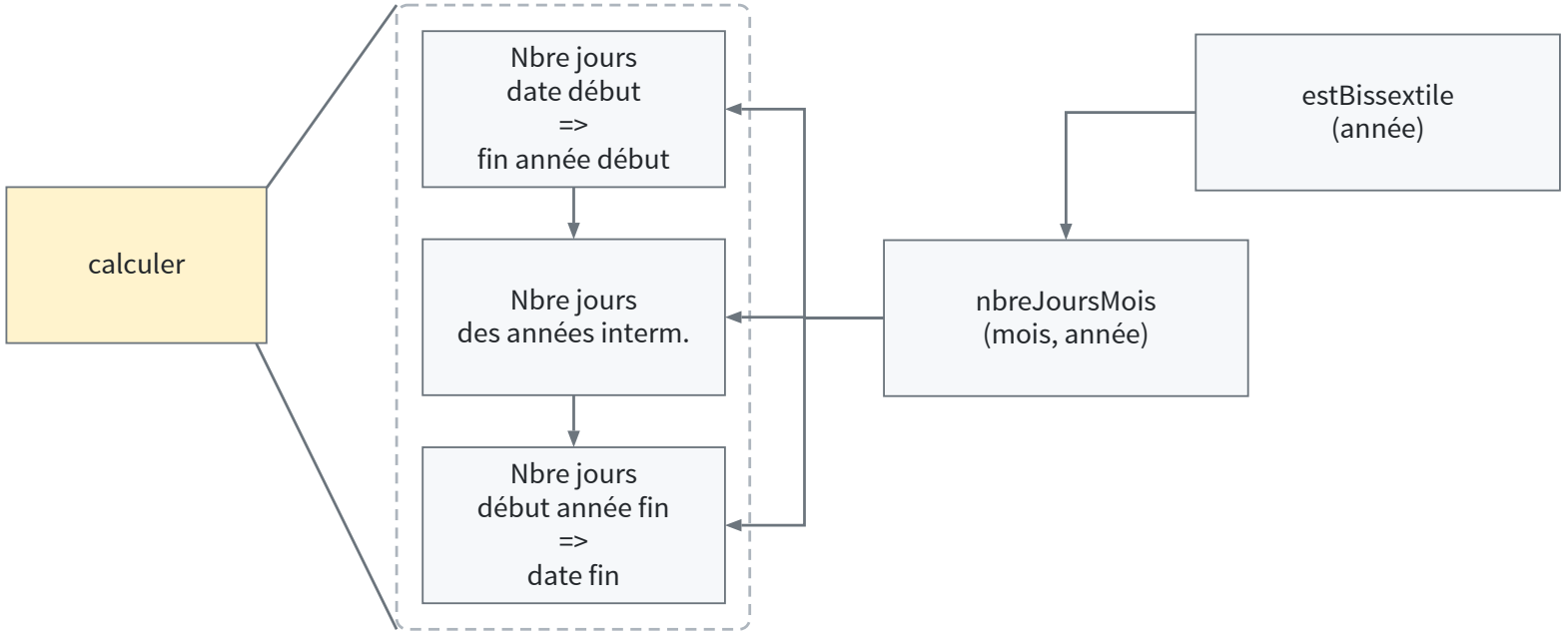
Octobre 2026

L	M	M	J	V	S	D
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

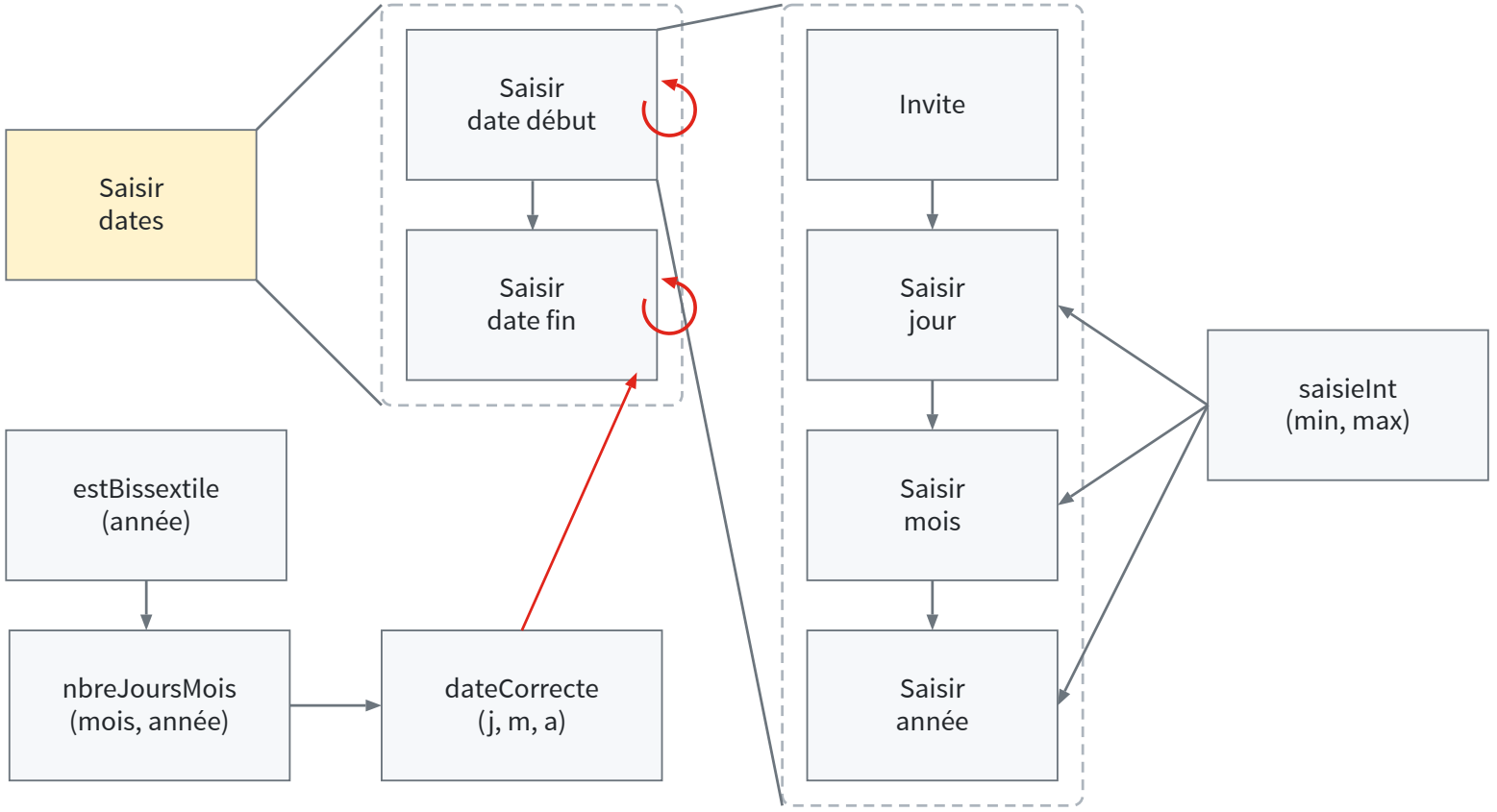
Approche descendante



Approche descendante



Approche descendante



Approche descendante

- Cette analyse nous a permis d'identifier quelques fonctions :
  - `saisirDate` retourne (ou rend par paramètres) une date
  - `calculer` retourne le nombre de jours entre deux dates
  - `afficher` affiche le nombre de jours dans un format choisi
  - `saisieInt` saisit un entier dans un intervalle choisi
  - `nbreJoursMois` retourne le nombre de jours pour un mois et une année
  - `estBissextile` retourne un booléen indiquant si l'année est bissextile
  - `dateCorrecte` retourne un booléen indiquant si la date est correcte
  - …

Des fonctions de quelle longueur ?



- L'approche descendante pose une question : jusqu'à quel niveau de détail faut-il décomposer les tâches ? **Quelle doit être la longueur de nos fonctions ?**
- Des fonctions **trop longues** sont difficiles à comprendre, voire à lire, si elles ne tiennent pas sur un écran.
- Des fonctions **trop courtes** augmentent sensiblement le travail additionnel de structuration, car chaque fonction doit être :
  - conçue pour être réutilisable
  - déclarée
  - définie et codée
  - testée
  - commentée

Cpp Core Guidelines

isocpp.github.io/CppCoreGuidelines

- F.1 choisir un nom de fonction aussi pertinent que possible.
- F.2 une fonction doit faire une opération logique simple.
- F.3 une fonction doit être courte et simple.
- F.8 préférer des fonctions « pures » (résultat prévisible, pas d'effet de bord, pas d'affichage, …).
- F.10 si une opération peut être réutilisée, nommez-la.
- F.16 passer les paramètres d'entrée de la manière la plus économe possible :

```
void f1(const string& s);  // OK: pass by reference to const; always cheap
void f2(string s);         // bad: potentially expensive
void f3(int x);            // OK: Unbeatable
void f4(const int& x);     // bad: overhead on access in f4()
```

Cpp Core Guidelines



- F.17 passer les paramètres d'entrée-sortie par référence non constante :

```
void update(Record& r); // assume that update writes to r
```

- F.20 pour les valeurs de sortie, préférer un **`return`**.
- F.43 ne jamais retourner un pointeur ou une référence à un objet local :

```
int* f() {
   int fx = 9;
   return &fx;
}
```

✘

```
int& f() {
   int fx = 9;
   return fx;
}
```

✘

- F.56 éviter les conditions imbriquées inutiles.



9. Surcharge de fonctions

Surcharge de fonctions

- **Surcharger** une fonction, c'est utiliser le **même nom** pour des fonctions que l'on **distingue** uniquement par leurs **paramètres**.
- La signature d'une fonction correspond aux **caractéristiques de ses paramètres** :
  - Leur nombre et leur ordre
  - Leurs types, y compris la méthode de passage (valeur, référence ou référence constante)
- Le **compilateur choisira** la fonction à utiliser **selon les paramètres effectifs** (de l'appel) **par rapport aux paramètres formels** (du prototype) des fonctions candidates.



Surcharge de fonctions



- Les éléments suivants **ne permettent pas de différencier deux surcharges** :
  - Le type de **retour**
  - Les **valeurs par défaut** des paramètres
  - La présence d'un `const` pour un paramètre passé **par valeur**
- Si deux fonctions de même nom ne diffèrent que par l'un ou plusieurs éléments ci-dessus, le **compilateur signale une erreur à la déclaration** de la deuxième fonction :

```
int add(int lhs, int rhs=0) {
   return lhs + rhs;
}
```

```
short add(const int lhs, const int rhs) {
   return short(lhs + rhs);
}
```

```
error: functions that differ only in their return type cannot be overloaded
```

HEIG VD



Surcharge de fonctions : exemple

- La même opération, offerte pour plusieurs types : le nom dit ce que la fonction fait, le paramètre dit sur quoi.
- Le compilateur choisit la version dont le paramètre correspond au type de l'argument.
- Bien utilisée, la surcharge se lit sans hésitation : chaque appel correspond évidemment à une version.

```
void afficher(int n)           { cout << "entier : " << n; }
void afficher(double x)        { cout << "reel   : " << x; }
void afficher(const string& s) { cout << "texte  : " << s; }
afficher(42);        // -> version int
afficher(3.14);      // -> version double
afficher("Hello");   // -> version const string& (conversion)
```

Surcharge de fonctions : ambiguïté



- Si aucune version ne correspond mieux que les autres, l'appel est ambigu : c'est une erreur de compilation, jamais un choix arbitraire du compilateur.
- Pour lever le doute : un suffixe de type pour un littéral, **`static_cast`** pour une variable.
- Le meilleur remède reste de choisir des surcharges clairement distinctes.

```
void afficher(long n)   { cout << "entier long : " << n; }
void afficher(double x) { cout << "reel        : " << x; }
int valeur = 42;

// int -> long ? int -> double ? AMBIGU : erreur de compilation
afficher(valeur);
// litteral avec suffixe -> version long
afficher(42L);
// conversion explicite -> version double
afficher(static_cast<double>(valeur));
```

HEIG VD



En résumé

- Le compilateur choisit la surcharge dont les paramètres correspondent le mieux aux arguments de l'appel :
  - correspondance exacte des types en premier
  - sinon, conversions légères (ajout de `const`, petites promotions numériques)
  - sinon, autres conversions (avec perte possible)
- Si deux surcharges conviennent aussi bien : appel ambigu, erreur de compilation.
- En pratique : choisissez des surcharges clairement distinctes ; en cas de doute, le compilateur vous le dira.